

Divide and Conquer

Algorithms and Complexity - Computer Science

1. The three steps

Divide the problem into smaller instances of the same problem, conquer each by solving it recursively, and combine the partial results into an answer for the original. The pattern applies whenever subproblems are independent and combining them is cheaper than solving the whole directly.

2. Worked examples

Merge sort splits the array in half, sorts each half, and merges the two sorted runs in linear time. Binary search discards half the search space per comparison. Quicksort partitions around a pivot so that combining is free, moving the work into the divide step instead.

3. Recurrences and the master theorem

A divide-and-conquer algorithm is described by $T(n) = aT(n/b) + f(n)$, where a is the number of subproblems, n/b their size, and $f(n)$ the cost of dividing and combining. The master theorem compares $f(n)$ with $n^{\log_b a}$: whichever dominates sets the result, and a tie introduces a logarithmic factor. Merge sort gives $T(n) = 2T(n/2) + O(n)$, which resolves to $\Theta(n \log n)$.

4. When it does not apply

Overlapping subproblems make plain recursion re-solve the same instance repeatedly - naive Fibonacci is exponential for this reason. That case calls for dynamic programming, which memoises results rather than recomputing them.